GUÍA PRÁCTICA LAB 04 Y PROMPTS OPTIMIZADOS • EDICIÓN TELECOMUNICACIONES (TELECOM NEXUS)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 04 (ES)

Laboratorio 04: Capstone Multi-Agente OSS/BSS — Google ADK, Protocolo A2A, ADK Dev UI (`adk web`), Terraform y Gemini Enterprise

English | **Español** | Prompts (EN) | Prompts (ES)

1. Escenario Comercial y Técnico de Telecomunicaciones

La Mesa de Ventas Empresariales B2B tiene **15 órdenes pendientes TM Forum TMF622 de Network Slicing 5G** (`ORD-TMF622-9001` a `ORD-TMF622-9015`) que representan millones en Valor Anual de Contrato (ACV). Antes de aprovisionar un slice en un Código de Área de Seguimiento (TAC), la plataforma debe coordinar **dos dominios especializados de ingeniería de telecomunicaciones**:

1. **Dominio Técnico OSS (`RANSliceCapacityAgent`)**: Evalúa las celdas 5G RAN bajo 3GPP TS 28.552 en el TAC objetivo (`prb_utilization_pct < 80.0`, `pdcp_latency_ms <= max_latency_ms`, `available_bandwidth_mbps >= requested_bandwidth_mbps`). Si la celda primaria está congestionada (`>= 80%` PRB) y existe una celda vecina saludable en `neighbor_cells`, puede ejecutar **remediación autónoma de slice** (direccionamiento de tráfico a celda vecina + reasignación de PRB).
2. **Dominio Comercial BSS (`BSSCommercialSLAAgent`)**: Cuantifica el Valor Anual de Contrato protegido (`MRR * 12`), la exposición evitada a penalizaciones SLA (`MRR * 2`), los incumplimientos regionales activos de SLA B2B y la fuga de ingresos en CDRs de roaming GSMA TAP3.12.

En este laboratorio capstone, utilizará **Google Antigravity** (y `/plan` o `/teamwork`) para:

- Implementar el **Motor de Factibilidad de Órdenes TMF622 y Remediación de Slices** (`src/services/order_feasibility.py`).
- Construir un **Orquestador Multi-Agente con Google ADK (Agent Development Kit)** (`src/agents/adk_a2a_orchestrator.py` y `adk_agents/telecom_nexus/agent.py`) coordinando `RANSliceCapacityAgent` y `BSSCommercialSLAAgent`.
- Exponer el sistema multi-agente mediante el **Protocolo A2A (Agent2Agent)** (`GET /.well-known/agent.json` Agent Card y `POST /api/v1/a2a` JSON-RPC 2.0 `tasks/send`).
- Probar su grafo multi-agente de forma interactiva en su laptop usando la **Interfaz de Desarrollo ADK (`adk web`)** (`./scripts/run_adk_web.sh` en `http://localhost:8080`).
- Validar el despliegue en la nube con **Terraform (`infra/terraform/`)** y registrar el agente A2A en **Google Gemini Enterprise** (`./deploy/deploy_gcp.sh --dry-run`).

2. Arquitectura Multi-Agente ADK + A2A + Gemini Enterprise

3. Instrucciones Paso a Paso

Paso 1: Inicializar el Estado Inicial del Laboratorio 04

```
./labs/lab-04-multi-agent-capstone/install.sh
```

Esto asegura que las soluciones de los Laboratorios 01–03 estén activas y establece `src/services/order_feasibility.py` y `src/agents/adk_a2a_orchestrator.py` en sus plantillas iniciales del Laboratorio 04.

Paso 2: Implementar Factibilidad de Órdenes TMF622 y Direccionamiento Autónomo de Slices

Abra PROMPTS.es.md y pegue el **Prompt 1** en Antigravity para implementar `evaluate_order_feasibility()` y `remediate_slice_for_order()` en `src/services/order_feasibility.py`.

Paso 3: Construir el Orquestador Multi-Agente Google ADK y el Servidor de Protocolo A2A

Pegue el **Prompt 2** de PROMPTS.es.md en Antigravity para completar `src/agents/adk_a2a_orchestrator.py`:

- `RANSliceCapacityAgent.assess_order_ran_slice(order_id, auto_remediate)`
- `BSSCommercialSLAAgent.assess_commercial_impact(order_id, oss_result)`
- `TelecomNexusOrchestratorAgent.run_order_assessment(order_id, auto_remediate)`
- `get_a2a_agent_card(base_url)` (`GET /.well-known/agent.json`)
- `handle_a2a_jsonrpc_request(payload)` (`POST /api/v1/a2a`)

Paso 4: Pruebas Locales Interactivas con la Interfaz de Desarrollo ADK (`adk web`)

Inicie la **Interfaz de Desarrollo de Google ADK** localmente para inspeccionar su `root_agent` (`telecom_nexus`), los subagentes (`ran_slice_capacity_agent`, `bss_commercial_sla_agent`, `a2a_protocol_bridge_agent`), llamadas a herramientas y transiciones de estado:

```
# Validar el paquete de agentes ADK y la configuración de conexión a Gemini Enterprise:
./scripts/run_adk_web.sh --check

# Iniciar la interfaz interactiva ADK Web UI en http://localhost:8080:
./scripts/run_adk_web.sh
```

Dentro de `http://localhost:8080`, seleccione **`telecom_nexus`** en el menú desplegable de agentes y pruebe consultas como:

- *"Evalúa la orden TMF622 ORD-TMF622-9001 usando el Agente de Capacidad RAN y el Agente Comercial SLA BSS."*

- *"Ejecuta remediación autónoma de slice 5G para ORD-TMF622-9001 y calcula el Valor Anual de Contrato protegido."*
- *"Envía una tarea JSON-RPC A2A para ORD-TMF622-9003."*

Paso 5: Desplegar con Terraform y Registrar en Google Gemini Enterprise

Revise la configuración de Terraform en `infra/terraform/` (`versions.tf`, `variables.tf`, `main.tf`, `outputs.tf`) y el script de registro A2A en Gemini Enterprise en `deploy/register_gemini_enterprise_agent.sh`.

Ejecute la validación local en modo `dry-run` (sin requerir credenciales activas de GCP):

```
./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run
```

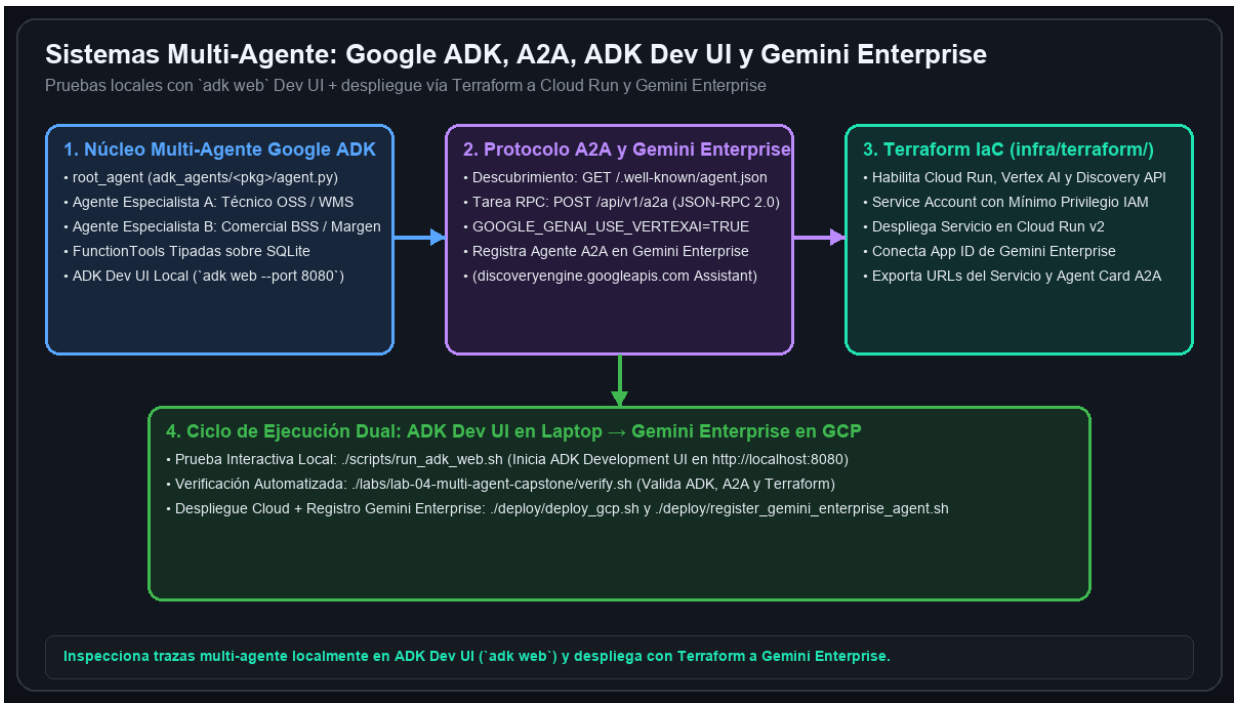
Para desplegar en un proyecto real de Google Cloud con Gemini Enterprise:

```
export GOOGLE_CLOUD_PROJECT="su-proyecto-telecom-gcp"
export GEMINI_ENTERPRISE_APP_ID="telecom-nexus-enterprise-app"
./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --project "$GOOGLE_CLOUD_PROJECT" --gem \
  ini-app-id "$GEMINI_ENTERPRISE_APP_ID"
```

Paso 6: Verificar su Solución del Laboratorio 04

```
# Opcional: Aplicar la solución de referencia
./labs/lab-04-multi-agent-capstone/apply_solution.sh

# Ejecutar la suite completa de 21 pruebas de verificación de los Labs 01–04
./labs/lab-04-multi-agent-capstone/verify.sh
```



LAB 04 PROMPTS OPTIMIZADOS PARA COPIAR Y PEGAR (ES)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Prompts de Antigravity del Laboratorio 04: Multi-Agente ADK, Protocolo A2A, ADK Dev UI y Gemini Enterprise

English | **Español** | Guía del Laboratorio 04

Prompt 1: Implementar Factibilidad de Órdenes TMF622 y Remediación Autónoma de Slices 5G

Copie y pegue este prompt en Google Antigravity (también puede anteponer `/plan` para planificación arquitectónica paso a paso):

```
Inspecciona `src/services/order_feasibility.py`, `src/models.py` y `tests/test_lab04_cap \
  stone.py`.

Implementa `evaluate_order_feasibility(order_id)` y `remediate_slice_for_order(order_id, \
   steering_mode)` en `src/services/order_feasibility.py` siguiendo todas las reglas de  \
  `GEMINI.md` y `.agents/rules/telecom-standards.md`:

1. En `evaluate_order_feasibility(order_id)`:
   - Consulta `sales_orders` por `order_id` (lanza `ValueError` si no existe).
   - Consulta `ran_telemetry` para las celdas que coincidan con `tac = target_tac` y `s_ \
  nssai_sst = requested_slice_sst`.
   - Una celda es factible si `prb_utilization_pct < 80.0`, `pdcp_latency_ms <= max_late \
  ncy_ms` y `available_bandwidth_mbps >= requested_bandwidth_mbps`.
   - Si una celda congestionada (`prb_utilization_pct >= 80.0` o `pdcp_latency_ms > max_ \
  latency_ms`) tiene una celda vecina factible en `neighbor_cells`, devuelve `feasibilit \
  y_status = "REQUIRES_SLICE_REMEDIATION"`, `requires_remediation = True`, `congested_ce \
  ll_id` y `recommended_cell_id` (la celda vecina).
   - De lo contrario, si existe una celda factible sin congestión, devuelve `feasibility \
  _status = "READY_TO_ACTIVATE"`, `requires_remediation = False` y `recommended_cell_id` \
  .
   - Si ninguna celda puede satisfacer los requisitos de ancho de banda/latencia, devuel \
  ve `feasibility_status = "REJECTED"`.
   - Calcula `protected_annual_revenue_usd` como `monthly_recurring_revenue_usd * 12` us \
  ando `Decimal`.

2. En `remediate_slice_for_order(order_id, steering_mode)`:
   - Verifica que la orden esté actualmente en estado `REQUIRES_SLICE_REMEDIATION` (de l \
  o contrario lanza `ValueError`).
   - Descarga tráfico de la celda congestionada hacia la celda vecina objetivo en `ran_t \
  elemetry` (reduciendo `prb_utilization_pct` de la celda congestionada a `58.5%`, reduc \
  iendo su `pdcp_latency_ms` y actualizando `sales_orders.status = 'READY_TO_ACTIVATE'`  \
  y `sales_stage = 'PROVISIONED'`).
   - Devuelve el resumen de remediación incluyendo `protected_annual_revenue_usd` (`MRR  \
  * 12`), `avoided_sla_penalty_usd` (`MRR * 2`) y `total_commercial_value_preserved_usd` \
   (`MRR * 14`).
```

Prompt 2: Diseñar el Sistema Multi-Agente Google ADK y los Endpoints del Protocolo A2A

Copie y pegue este prompt en Google Antigravity (o use `/teamwork` para coordinar el diseño de los agentes OSS y BSS):

```
Inspecciona `src/agents/adk_a2a_orchestrator.py`, `adk_agents/telecom_nexus/agent.py`, ` \
  src/main.py` y `tests/test_lab04_capstone.py`.

Completa `src/agents/adk_a2a_orchestrator.py` para implementar nuestro Sistema Multi-Age \
  nte con Google ADK y el servidor del Protocolo A2A (Agent2Agent):

1. `RANSliceCapacityAgent`:
   - Implementa `assess_order_ran_slice(order_id, auto_remediate=False)` usando `evaluat \
  e_order_feasibility` y (cuando `auto_remediate=True` y el estado sea `REQUIRES_SLICE_R \
  EMEDIATION`) `remediate_slice_for_order`.
   - Devuelve `agent_name = "ran_slice_capacity_agent"`, `initial_feasibility_status`, ` \
  final_feasibility_status`, `target_tac`, `recommended_cell_id` y `auto_remediation_exe \
  cuted`.

2. `BSSCommercialSLAAgent`:
   - Implementa `assess_commercial_impact(order_id, oss_result)` consultando `sales_orde \
  rs`, `evaluate_contract_sla_compliance()` y `audit_cdr_revenue_leakage()`.
   - Calcula `annual_contract_value_usd` (`MRR * 12`) y `avoided_sla_penalty_risk_usd` ( \
  `MRR * 2`) con `Decimal`.
   - Asigna `final_feasibility_status` a `commercial_decision`:
     - `READY_TO_ACTIVATE` -> `APPROVE_IMMEDIATELY`
     - `REQUIRES_SLICE_REMEDIATION` -> `APPROVE_WITH_AUTOMATED_RAN_STEERING`
     - `REJECTED` -> `HOLD_REQUIRES_RAN_CAPACITY_CAPEX`

3. `TelecomNexusOrchestratorAgent`:
   - Coordina `RANSliceCapacityAgent` (Paso 1) y `BSSCommercialSLAAgent` (Paso 2) usando \
   Google ADK (jerarquía `SequentialAgent` / `LlmAgent` de `google-adk` con ejecución lo \
  cal determinista).
   - Devuelve `orchestrator_name`, `adk_architecture`, `oss_technical_assessment`, `bss_ \
  commercial_assessment`, `executive_decision` y `agent_trace`.

4. Integración del Protocolo A2A:
   - Implementa `get_a2a_agent_card(base_url)` devolviendo la Agent Card para `TelecomNe \
  xus-OSS-BSS-Orchestrator` con las habilidades (`skills`) `tmf622_order_feasibility`, ` \
  autonomous_slice_remediation` y `sla_revenue_assurance`.
   - Implementa `handle_a2a_jsonrpc_request(payload)` soportando JSON-RPC 2.0 `"tasks/se \
  nd"` y `"message/send"`, extrayendo `order_id` y `auto_remediate` de las partes del me \
  nsaje A2A y devolviendo una Tarea A2A completada con `status.state = "completed"` y `a \
  rtifacts`.

Finalmente, ejecuta `./.venv/bin/pytest tests/test_lab04_capstone.py -v` y `./scripts/ru \
  n_adk_web.sh --check` para verificar que todas las pruebas pasen.
```

Prompt 3: Probar con ADK Web UI (`adk web`) y Validar el Despliegue en Terraform + Gemini Enterprise

```
1. Valida el paquete de agentes para ADK Web UI (`adk_agents/telecom_nexus/agent.py`) y  \
  la configuración de Discovery Engine para Gemini Enterprise ejecutando:
   ./scripts/run_adk_web.sh --check

2. Valida nuestro Dockerfile, la infraestructura de Terraform (`infra/terraform/`) y el  \
```

payload de registro del agente A2A en Gemini Enterprise (`deploy/register_gemini_enter \
prise_agent.sh`) en modo dry-run local:
```
./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run
```

3. Ejecuta `./labs/lab-04-multi-agent-capstone/verify.sh` para confirmar que las 21 prue \
bas de los Laboratorios 01, 02, 03 y 04 pasan al 100%.